

Name : Vivek Singh
RegNumber : 230905408
Roll Number: A50

Lab 8: Programs on Matrix in CUDA

Q1).

Code:

```
#include <stdio.h>
#define N 4

// (a) One thread per ROW
__global__ void add_row(int *A, int *B, int *C) {
    int row = threadIdx.x;
    if (row < N) {
        for (int j = 0; j < N; j++) {
            C[row*N + j] = A[row*N + j] + B[row*N + j];
        }
    }
}

// (b) One thread per COLUMN
__global__ void add_col(int *A, int *B, int *C) {
    int col = threadIdx.x;
    if (col < N) {
        for (int i = 0; i < N; i++) {
            C[i*N + col] = A[i*N + col] + B[i*N + col];
        }
    }
}

// (c) One thread per ELEMENT
__global__ void add_elem(int *A, int *B, int *C) {
    int i = blockIdx.x;
    int j = threadIdx.x;

    if (i < N && j < N) {
        C[i*N + j] = A[i*N + j] + B[i*N + j];
    }
}

int main() {
    int A[N*N], B[N*N];
    int C1[N*N], C2[N*N], C3[N*N];

    int *dA, *dB, *dC1, *dC2, *dC3;

    // Initialize input
    for(int i = 0; i < N*N; i++) {
        A[i] = i;
        B[i] = i;
    }

    // Allocate device memory
    cudaMalloc(&dA, N*N*sizeof(int));
```

```

cudaMalloc(&dB, N*N*sizeof(int));
cudaMalloc(&dC1, N*N*sizeof(int));
cudaMalloc(&dC2, N*N*sizeof(int));
cudaMalloc(&dC3, N*N*sizeof(int));

// Copy to device
cudaMemcpy(dA, A, N*N*sizeof(int), cudaMemcpyHostToDevice);
cudaMemcpy(dB, B, N*N*sizeof(int), cudaMemcpyHostToDevice);

// ---- RUN ALL KERNELS ----

add_row<<<1, N>>>(dA, dB, dC1);
add_col<<<1, N>>>(dA, dB, dC2);
add_elem<<<N, N>>>(dA, dB, dC3);

cudaDeviceSynchronize();

// Copy results back
cudaMemcpy(C1, dC1, N*N*sizeof(int), cudaMemcpyDeviceToHost);
cudaMemcpy(C2, dC2, N*N*sizeof(int), cudaMemcpyDeviceToHost);
cudaMemcpy(C3, dC3, N*N*sizeof(int), cudaMemcpyDeviceToHost);

// ---- PRINT RESULTS ----

printf("\nRow-wise Addition:\n");
for(int i = 0; i < N*N; i++) {
    printf("%d ", C1[i]);
    if((i+1)%N == 0) printf("\n");
}

printf("\nColumn-wise Addition:\n");
for(int i = 0; i < N*N; i++) {
    printf("%d ", C2[i]);
    if((i+1)%N == 0) printf("\n");
}

printf("\nElement-wise Addition:\n");
for(int i = 0; i < N*N; i++) {
    printf("%d ", C3[i]);
    if((i+1)%N == 0) printf("\n");
}

// Free memory
cudaFree(dA);
cudaFree(dB);
cudaFree(dC1);
cudaFree(dC2);
cudaFree(dC3);

return 0;
}

```

Output :

```
Row-wise Addition:
```

```
0 2 4 6
8 10 12 14
16 18 20 22
24 26 28 30
```

```
Column-wise Addition:
```

```
0 2 4 6
8 10 12 14
16 18 20 22
24 26 28 30
```

```
Element-wise Addition:
```

```
0 2 4 6
8 10 12 14
16 18 20 22
24 26 28 30
```

```
STUDENT@MIT-ICT-LAB5-15:~/Desktop/230905408/lab8$
```

Q2)

Code:

```
#include <stdio.h>
#define N 3

// (a) One thread per ROW
__global__ void mul_row(int *A, int *B, int *C) {
    int row = threadIdx.x;

    if (row < N) {
        for (int j = 0; j < N; j++) {
            int sum = 0;
            for (int k = 0; k < N; k++) {
                sum += A[row*N + k] * B[k*N + j];
            }
            C[row*N + j] = sum;
        }
    }
}

// (b) One thread per COLUMN
__global__ void mul_col(int *A, int *B, int *C) {
    int col = threadIdx.x;

    if (col < N) {
        for (int i = 0; i < N; i++) {
            int sum = 0;
            for (int k = 0; k < N; k++) {
                sum += A[i*N + k] * B[k*N + col];
            }
            C[i*N + col] = sum;
        }
    }
}

// (c) One thread per ELEMENT
__global__ void mul_elem(int *A, int *B, int *C) {
    int i = blockIdx.x;
    int j = threadIdx.x;

    if (i < N && j < N) {
        int sum = 0;
        for (int k = 0; k < N; k++) {
            sum += A[i*N + k] * B[k*N + j];
        }
        C[i*N + j] = sum;
    }
}

int main() {
    int A[N*N], B[N*N];
    int C1[N*N], C2[N*N], C3[N*N];

    int *dA, *dB, *dC1, *dC2, *dC3;

    // Initialize matrices
```

```

printf("Matrix A:\n");
for(int i = 0; i < N*N; i++) {
    A[i] = i + 1;
    printf("%d ", A[i]);
    if((i+1)%N == 0) printf("\n");
}

printf("\nMatrix B:\n");
for(int i = 0; i < N*N; i++) {
    B[i] = i + 1;
    printf("%d ", B[i]);
    if((i+1)%N == 0) printf("\n");
}

// Allocate device memory
cudaMalloc(&dA, N*N*sizeof(int));
cudaMalloc(&dB, N*N*sizeof(int));
cudaMalloc(&dC1, N*N*sizeof(int));
cudaMalloc(&dC2, N*N*sizeof(int));
cudaMalloc(&dC3, N*N*sizeof(int));

// Copy to device
cudaMemcpy(dA, A, N*N*sizeof(int), cudaMemcpyHostToDevice);
cudaMemcpy(dB, B, N*N*sizeof(int), cudaMemcpyHostToDevice);

// Launch kernels
mul_row<<<1, N>>>(dA, dB, dC1);
mul_col<<<1, N>>>(dA, dB, dC2);
mul_elem<<<N, N>>>(dA, dB, dC3);

cudaDeviceSynchronize();

// Copy back results
cudaMemcpy(C1, dC1, N*N*sizeof(int), cudaMemcpyDeviceToHost);
cudaMemcpy(C2, dC2, N*N*sizeof(int), cudaMemcpyDeviceToHost);
cudaMemcpy(C3, dC3, N*N*sizeof(int), cudaMemcpyDeviceToHost);

// Print results
printf("\nRow-wise Multiplication:\n");
for(int i = 0; i < N*N; i++) {
    printf("%d ", C1[i]);
    if((i+1)%N == 0) printf("\n");
}

printf("\nColumn-wise Multiplication:\n");
for(int i = 0; i < N*N; i++) {
    printf("%d ", C2[i]);
    if((i+1)%N == 0) printf("\n");
}

printf("\nElement-wise Multiplication:\n");
for(int i = 0; i < N*N; i++) {
    printf("%d ", C3[i]);
    if((i+1)%N == 0) printf("\n");
}

// Free memory
cudaFree(dA);
cudaFree(dB);

```

```
    cudaFree(dC1);  
    cudaFree(dC2);  
    cudaFree(dC3);  
  
    return 0;  
}
```

Output:

```
STUDENT@MIT-ICT-LAB5-15:~/Desktop/230905408/lab8$ ./q2  
Matrix A:  
1 2 3  
4 5 6  
7 8 9  
  
Matrix B:  
1 2 3  
4 5 6  
7 8 9  
  
Row-wise Multiplication:  
30 36 42  
66 81 96  
102 126 150  
  
Column-wise Multiplication:  
30 36 42  
66 81 96  
102 126 150  
  
Element-wise Multiplication:  
30 36 42  
66 81 96  
102 126 150  
STUDENT@MIT-ICT-LAB5-15:~/Desktop/230905408/lab8$
```